

SAV

מ מעוצבים · פלטפורמת מסחר והתאמה אישית

High-Level Design Docu

Phase 1 (MVP) — ערכת



1. רקע ומטרת המוצר

Savor היא פלטפורמת מסחר והתאמה אישית של מטבחים מעוצבים. הלקוח יכול לבחור מטבח מוכן מקטלוג (לפי דגם, גודל וצבע), להוסיף מוצרים משלימים (כיורים, ברזים, ידיות, שיש), ובעתיד (Phase 2) גם לתכנן את המטבח שלו בעצמו באמצעות כלי תכנון אינטראקטיבי עם תצוגת 2D/3D וחישוב מחיר אוטומטי.

מטרת המוצר: לאפשר עיצוב ורכישה של מטבח איכותי בצורה פשוטה, נגישה ואישית, עם חוויית בחירה ותכנון אונליין שמקטינה חוסר ודאות לפני רכישה.

מסמך זה מתאר את ה-High Level Design של המערכת בשלב ה-MVP (Phase 1), ואת ההשפעות הארכיטקטוניות של מודול ההתאמה האישית (Phase 2) שעדיין לא ייבנה בשלב זה אך חייב להילקח בחשבון כדי שהאינטגרציה העתידית תהיה חלקה.

2. Scope

Phase 1 — MVP 2.1

תחום	כלול
קטלוג מטבחים	כן — לפי גודל, דגם וצבע ✓
עמוד מוצר	כן — תיאור, מחיר, תמונות, מוצרים משלימים ✓
מוצרים משלימים	כן — כיורים, ברזים, ידיות, שיש ✓
עגלת קניות (Cart)	כן ✓
Checkout ותשלום	כן — אינטגרציה עם Stripe / Tranzila ✓
ניהול הזמנות (Order lifecycle)	כן — בסיסי ✓
Wishlist	כן ✓
דפי תוכן (אודות, צור קשר, מדיניות)	כן — CMS-light ✓
תמיכה Desktop + Mobile (responsive)	כן ✓

2.2 Phase 2 — מודול ההתאמה האישית (Configurator)

⚠ מחוץ ל-Scope, אך משפיע על העיצוב

מודולים אלו לא ייבנו ב-Phase 1, אך ארכיטקטורת Phase 1 חייבת להתחשב בהם.

תחום	כלול ב-Phase 2
תכנון אינטראקטיבי (drag & drop)	✓
תצוגת 2D/3D	✓
חישוב מחיר אוטומטי בזמן תכנון	✓
הדמיית AR	✓ (שיטת המימוש עדיין לא הוחלטה)

החלטה מרכזית

מודול ההתאמה האישית יבנה כ

מודול/שירות עצמאי

המתחבר למערכת Phase 1 דרך ממשקים מוגדרים (API contracts).

מודל הדאטה של Phase 1 חייב לכלול מהיום תמיכה ב"מוצר מורכב" (Composite Product)

— ראו סעיף 6.

2.3 Explicitly Out of Scope

- מערכת Admin / ניהול מלאי
- AR — בחירת טכנולוגיה (ייקבע בנפרד)
- Configurator engine עצמו (Phase 2)
- כל היבט הקשור ל-B2B / multi-tenancy

3. דרישות פונקציונליות עיקריות (Phase 1)

- קטלוג — עיון במטבחים מסוננים לפי גודל (לדוגמה "1.5 מטר") ולפי דגם/צבע (CREAM / CLOUD / LATTE).
- עמוד מוצר — תיאור מלא, מפרט (גוף ארון, חזיתות, גימור, פרזול), בחירת כמות, זמן אספקה (14 יום), קובץ הוראות התקנה להורדה, קרוסלת מוצרים משלימים.
- מוצרים משלימים — קטלוג נפרד עם פילטור לפי קטגוריה (ידיות, כיורים, שיש וכו').
- Wishlist — שמירת מוצרים לרשימת מועדפים, נגיש מכל מקום באתר.
- עגלת קניות — הוספה/הסרה/שינוי כמות, חישוב סכום כולל, מעבר לתשלום.
- Checkout — פרטי משלוח והרכבה, תשלום מאובטח מול ספק חיצוני.
- דפי תוכן — "אודות", "צור קשר" (כולל טופס פנייה), "מדריך ללקיחת מידה", "מדיניות אחריות והחזרה".

4. דרישות לא-פונקציונליות

קטגוריה	דרישה
Source of truth לתמחור	חישוב המחיר הסופי חייב להתבצע ולהיאומת בצד השרת בלבד. לקוח לא יכול לשנות מחיר ע"י מניפולציה בצד הלקוח.
Responsiveness	תמיכה מלאה ב-Mobile ו-Desktop (שתי גרסאות תצוגה).
ביצועים	טעינת קטלוג ועמוד מוצר מהירה — תמונות כבדות דורשות אופטימיזציית מדיה/CDN.
אבטחה	תשלומים — PCI compliance מואצל לספק התשלומים (לא לשמור פרטי כרטיס במערכת).
שפה/לוקאליזציה	עברית (RTL) כברירת מחדל.
הרחבתיות	מודל הדאטה והשירותים חייבים לתמוך בהוספת Configurator (Phase 2) בלי שינוי מבני גדול.
זמינות	תלוי בדרישות העסק — לציין SLA כשיתבהר.

5. ארכיטקטורה כללית (Phase 1)

5.1 גישה ארכיטקטונית

Backend יחיד מסוג **Monolith** (Python / FastAPI) המרכז את כל הלוגיקה העסקית של Phase 1: קטלוג, תמחור, עגלה, הזמנות, תשלומים. הבחירה ב-Monolith מתאימה לגודל הצוות וההיקף הנוכחי, ומפחיתה overhead תפעולי שלא נחוץ עדיין.

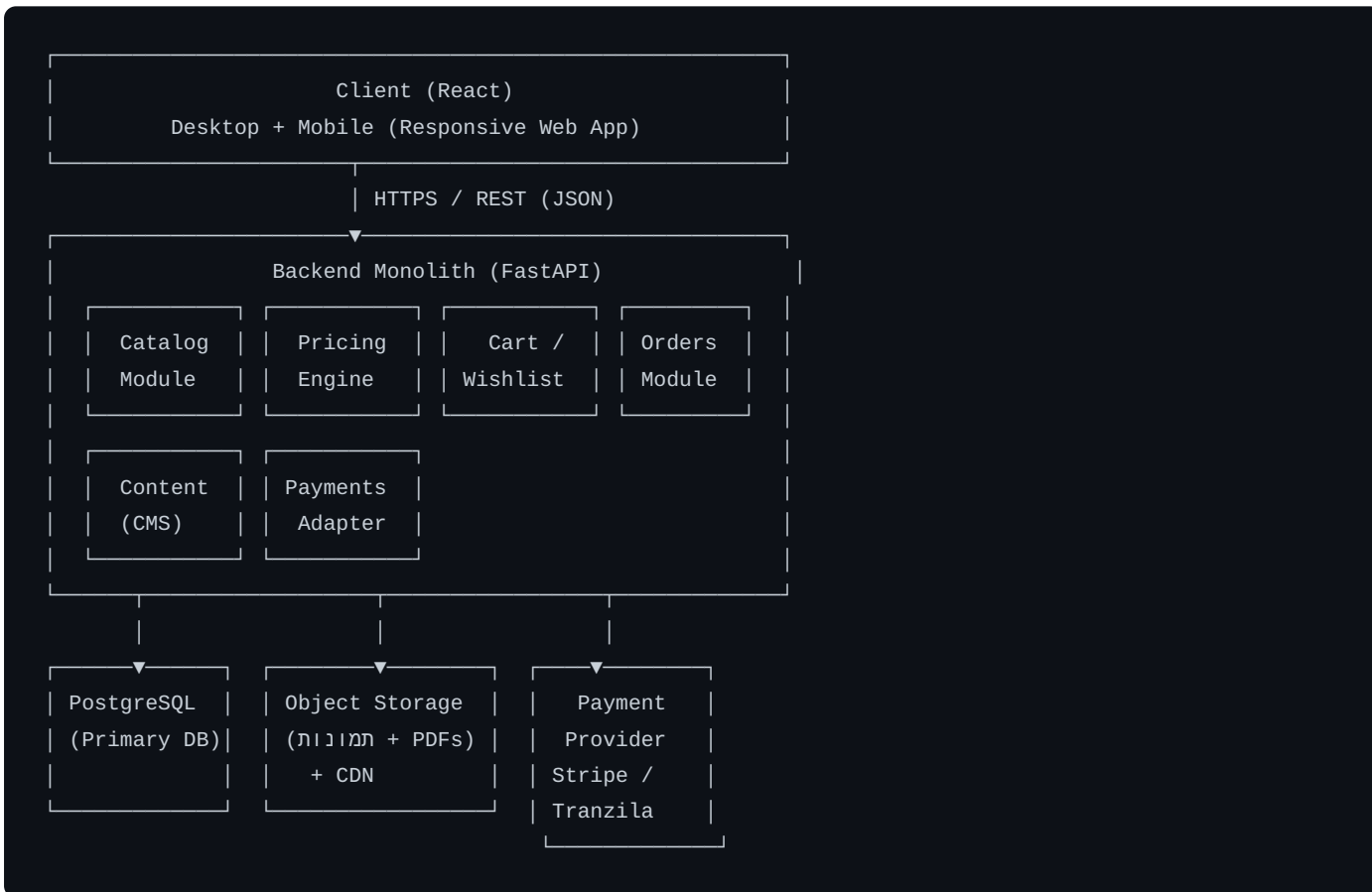
עיקרון מנחה לקראת Phase 2

ה-Monolith יבנה עם

הפרדת שכבות פנימית ברורה (Modular Monolith)

— כך שבעת הצורך ניתן יהיה לחלץ את "שירות התמחור" (Pricing Engine) לשירות עצמאי שגם ה-Configurator יוכל לצרוך, בלי refactor מקיף.

5.2 מרכיבי המערכת



5.3 תיאור המודולים

מודול	אחריות
Catalog Module	ניהול וחיפוש/סינון מוצרים (מטבחים + מוצרים משלימים), קטגוריות, תכונות (גודל, צבע, דגם).
Pricing Engine	

אחריות	מודול
מקור האמת היחיד לחישוב מחיר. מקבל "קונפיגורציה" של פריט/הזמנה ומחזיר מחיר מאומת. זהו המודול שיורחב ב-Phase 2 לתמיכה בחישוב מחיר דינמי.	
ניהול עגלת קניות ורשימת מועדפים לפי משתמש/session.	Cart / Wishlist
יצירת הזמנה מתוך עגלה, מעקב סטטוס (נוצרה → שולמה → בטיפול → נשלחה), שמירת snapshot של המחיר בזמן ההזמנה.	Orders Module
תוכן סטטי לעמודים. ניתן להתחיל כקבצים/טבלה פשוטה בלי CMS מלא.	-Content (CMS (light
שכבת הפשטה מול ספק תשלומים חיצוני — ניתן להחליף ספק בלי לשנות את שאר המערכת.	Payments Adapter

6. מודל דאטה — עיקרון מפתח: Composite Product

⚠️ זוהי הנקודה הקריטית ביותר לתאימות עתידית עם Phase 2

החלטות מודל הדאטה שיתקבלו עכשיו ישפיעו ישירות על עלות ה-migration בעתיד.

6.1 הבעיה

ב-prototype, "מטבח" מוצג כפריט קטלוג בודד, אבל בפועל הוא **מורכב מרכיבים**: יחידת תנור, ארון תחתון, חזיתות, צבע/גימור וכו'. אם נמדל זאת ב-Phase 1 כ"מוצר אטומי" פשוט (שורה אחת בטבלת מוצרים, מחיר קבוע), אזי כשנגיע ל-Phase 2 Configurator — שמייצר מטבח **דינמי** — נצטרך migration כבד של מודל הדאטה, ההזמנות הקיימות, והיסטוריית הרכישות.

6.2 העיקרון המוצע

לבנות את מודל המוצר כך שכל "מוצר נמכר" הוא בפועל:

- **Product Template** — ההגדרה הקטלוגית (לדוגמה "מטבח CREAM 1.5 מ"מ" כ"תבנית מוכנה מראש" עם רשימת רכיבים קבועה).
- **Product Configuration** — Snapshot קונקרטי של בחירה (אילו רכיבים, אילו צבעים, מידות) שמיוצר מתוך Template. ב-Phase 1 התבנית = הקונפיגורציה; ב-Phase 2 הלקוח יוכל לבנות קונפיגורציה משלו.
- **Line Item** — מצביע על Product Configuration + מחיר מאומת בזמן ההוספה לעגלה/הזמנה.

6.3 ישויות מרכזיות

ישות	תיאור
Product	פריט קטלוגי (מטבח/רכיב/מוצר משלים), עם metadata, תמונות, קטגוריה.
ProductComponent	רכיב בודד שמרכיב מוצר (יחידת תנור, חזית, צבע). ב-Phase 1 קשור סטטית למוצר; ב-Phase 2 ניתן לבחירה.
ProductConfiguration	אוסף קונקרטי של רכיבים + ערכי attributes (מידה, צבע) שמייצג "מטבח ספציפי שנבחר".
PriceRule	לוגיקת תמחור שמקבלת ProductConfiguration ומחזירה מחיר. ב-Phase 1 פשוטה (מחירים קבועים); ב-Phase 2 תורחב לתלות בגודל קיר, כמות רכיבים וכו'.
CartItem	ProductConfiguration + כמות + מחיר מאומת (locked at time of add).
Order / OrderItem	Snapshot קבוע לאחר checkout — לא תלוי בשינויים מאוחרים בקטלוג.

7. Frontend — עקרונות ארכיטקטוניים

- מבנה קלאסי e-commerce: קטלוג → עמוד מוצר → עגלה → checkout. **אין תלות ב-Three.js / react-three-fiber**
- **ב-Phase 1**
- מומלץ להפריד שכבת UI components (קלפי מוצר, עגלה, פילטרים) כך שיהיו reusable ב-Phase 2, לשמירה על שפה עיצובית אחידה.
- **State management** — עבור MVP פשוט מספיק React Context/Query (לדוגמה React Query לסנכרון מול ה-API) בלי Redux.

8. ממשק API בין Phase 1 ל-Phase 2 — הכנה לעתיד

כדי שמודול ההתאמה האישית יוכל להתחבר בלי לשבש את Phase 1, יש להגדיר כבר עכשיו את הנקודות הבאות כ"חוזה" עתידי:

1. Pricing Endpoint

POST /pricing/calculate

מקבל `ProductConfiguration` (רשימת רכיבים + attributes) ומחזיר מחיר מאומת. ב-Phase 1 קיים בצורה מצומצמת; ב-Phase 2 יורחב לקבל קונפיגורציות מורכבות. החתימה (signature) חייבת להיות מתוכננת לרhib ולא לשבור תאימות.

2. Cart Endpoint

POST /cart/items

הוספת `ProductConfiguration` לעגלה. Phase 2 יוסיף אפשרות להוסיף קונפיגורציה "מותאמת אישית" (custom), אבל מבנה הבקשה צריך להישאר עקבי.

3. Component Catalog API

Phase 2 יזדקק ל-endpoint שמחזיר את כל הרכיבים הזמינים (יחידות, צבעים, גדלים) עם מחיר ליחידה ו-constraints (אילו רכיבים מתאימים לאיזה גודל קיר). מומלץ ש-Catalog Module ב-Phase 1 ייבנה כך שהמידע הזה כבר קיים במודל, גם אם אין עדיין UI לחשוף אותו.

9. שאלות פתוחות והחלטות ממתכנת

#	נושא	סטטוס
1	טכנולוגיית AR (WebAR מול Native)	לא הוחלט — דורש בדיקת כיסוי דפדפנים ועומק חוויה נדרש
2	ספק תשלומים סופי (Stripe / Tranzila / אחר)	לא נסגר
3	מערכת Admin / ניהול מלאי	מחוץ ל-scope — ידרוש HLD נפרד
4	זמני SLA / זמינות	לא הוגדר
5	היקף ה-CMS לתוכן הסטטי	לא הוגדר — תלוי בכמות התוכן ותדירות העדכון

ה-(MVP (Phase 1 הוא חנות e-commerce סטנדרטית בהיקפה, אך מודל הדאטה שלה צריך להיבנות **מהיום** סביב הרעיון של "מוצר מורכב מרכיבים" כדי לא להתנגש עם דרישות ה-Configurator בעתיד. ארכיטקטורת ה-Backend כ-Modular Monolith מאפשרת גמישות זו בלי overhead מוקדם של microservices.

צעדים מומלצים להמשך

אישור מסמך זה ועדכון לפי הערות.

הגדרת סכימת דאטה מפורטת (DB schema) ל- `Product` / `ProductComponent` / `ProductConfiguration`.

הגדרת API Spec ראשוני (OpenAPI) למודולי הקטלוג, התמחור והעגלה.

החלטה על ספק תשלומים.

HLD נפרד למודול ה-Configurator (Phase 2) כשתתחיל העבודה עליו.